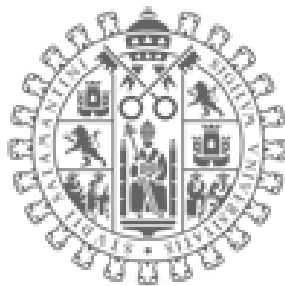


Calidad y Auditoría

Detección de errores, defectos y fallos de software



**VNiVERSiDAD
D SALAMANCA**

CAMPUS OF INTERNATIONAL EXCELLENCE

Miguel Barranquero Diez

Índice

1. Introducción	3
2. Programa 1	4
3. Programa 2	8
4. Programa 3	13
5. Programa 4	17
6. Programa 5	21

1. Introducción

El presente trabajo se centra en el análisis y la corrección de defectos en cinco programas software proporcionados. La estructura de este ejercicio se basa en la aplicación de una metodología de prueba y depuración específica para cada código fuente. Esta metodología implica:

- Identificar defectos.
- Identificar un caso de prueba que dé lugar a un fallo.
- Si es posible, identificar el caso de prueba que no dé lugar a un fallo.
- Si es posible, identificar el caso de prueba no produzca un estado de error.
- Si es posible, identificar el caso de prueba que produzca un estado de error pero no produzca un fallo.
- Corregir el defecto y verificar que el caso de prueba identificado en el punto 2 produce el resultado esperado

2. Programa 1

La Figure 1 corresponde con el primer programa a analizar:

```
public class Array1 {

    public static void main(String[] args) {

        //Devuelve el número de ocurrencias de 0 en x

        int x[]={1,0,0};
        int count=0;
        for (int i=1 ; i<x.length; i++)
        {
            if (x[i] == 0)
            {
                count ++;
            }
        }
        System.out.print ("Número de ceros: ");
        System.out.println(count);
    }

}
```

Figure 1: Programa 1

Identificar defectos (sin ejecutar el código)

El defecto del código es que el bucle for comienza desde el índice 1 ($i=1$), es decir, en el segundo elemento del array. Este debería de empezar en el índice 0 ($i=0$) para poder considerar y analizar todos los elementos del array.

```
for (int i=1 ; i<x.length; i++)
```

Figure 2: Programa 1 - Defecto

Identificar un caso de prueba que dé lugar a un fallo

Para realizar el caso de prueba que de lugar a fallo introduciré en la primera posición, ($i=0$ del array) un 0, este cero no se tendrá en cuenta por lo que el número de 0 esperado será incorrecto.

Para este ejemplo se ha declarado el array $x[] = \{0,0,0\}$ (Figure 3):

-Número de 0 esperado: 3.

-Número de 0 obtenido: 2.

```

1 public class Array1 {
2     public static void main(String[] args) {
3         //Devuelve el número de ocurrencias de 0 en x
4         int x[]={0,0,0};
5         int count=0;
6         for (int i=1 ; i<x.length; i++){
7             if (x[i] == 0){
8                 count ++;
9             }
10        }
11        System.out.print ("Número de ceros: ");
12        System.out.println(count);
13    }
14 }
15

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$ javac Array1.java
(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$ java Array1
Número de ceros: 2
(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$

```

Figure 3: Programa 2 - Fallo Ejemplo

Si es posible, identificar el caso de prueba que no dé lugar a un fallo

Para conseguir este caso de prueba bastará con que en la primera posición ($i=0$) del array no exista un 0, el defecto sigue estando pero con esta disposición (evitar 0 en la primera posición) la ejecución no dará fallo.

Para este ejemplo se ha declarado el array $x[] = \{1,0,0\}$ (Figure 4):

-Número de 0 esperado: 2.

-Número de 0 obtenido: 2.

```

1 public class Array1 {
2     public static void main(String[] args) {
3         //Devuelve el número de ocurrencias de 0 en x
4         int x[]={1,0,0};
5         int count=0;
6         for (int i=1 ; i<x.length; i++){
7             if (x[i] == 0){
8                 count ++;
9             }
10        }
11        System.out.print ("Número de ceros: ");
12        System.out.println(count);
13    }
14 }
15

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$ javac Array1.java
(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$ java Array1
Número de ceros: 2
(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$

```

Figure 4: Programa 1 - No Fallo Ejemplo

Si es posible, identificar el caso de prueba no produzca un estado de error

Para este caso de prueba será necesario que el array no tenga ningún 0, de esta manera aunque no se analice el primer elemento, no será relativo para determinar cuantos 0 hay en el array puesto que no existirá ninguno haciendo que no se produzca el estado de error.

Para este ejemplo se ha declarado el array $x[] = \{1,2,3\}$ (Figure 5):

-Número de 0 esperado: 0.

-Número de 0 obtenido: 0.

```

1 public class Array1 {
2     public static void main(String[] args) {
3         //Devuelve el número de ocurrencias de 0 en x
4         int x[]={1,2,3};
5         int count=0;
6         for (int i=1 ; i<x.length; i++){
7             if (x[i] == 0){
8                 count ++;
9             }
10        }
11        System.out.print ("Número de ceros: ");
12        System.out.println(count);
13    }
14 }
15

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas\$ javac Array1.java
(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas\$ java Array1
Número de ceros: 0
(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas\$

Figure 5: Programa 1 - No Fallo Ejemplo

Si es posible, identificar el caso de prueba que produzca un estado de error pero no produzca un fallo

Para este caso sera necesario que en el primer elemento nunca sea 0 para evitar el fallo, pero el estado de error seguirá ahí ya que no se evalúa la condición (cero/noCero) en el primer elemento (i=0).

Para este ejemplo se ha declarado el array $x[] = \{99,0,0\}$ (Figure 6):

-Número de 0 esperado: 2.

-Número de 0 obtenido: 2.

```

1 public class Array1 {
2     public static void main(String[] args) {
3         //Devuelve el número de ocurrencias de 0 en x
4         int x[]={99,0,0};
5         int count=0;
6         for (int i=1 ; i<x.length; i++){
7             if (x[i] == 0){
8                 count ++;
9             }
10        }
11        System.out.print ("Número de ceros: ");
12        System.out.println(count);
13    }
14 }
15

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas\$ javac Array1.java
(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas\$ java Array1
Número de ceros: 2
(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas\$

Figure 6: Programa 1 - No Fallo Ejemplo

Corregir el defecto y verificar que el caso de prueba identificado en el punto 2 produce el resultado esperado

Para corregir el defecto de cambia la línea 7:

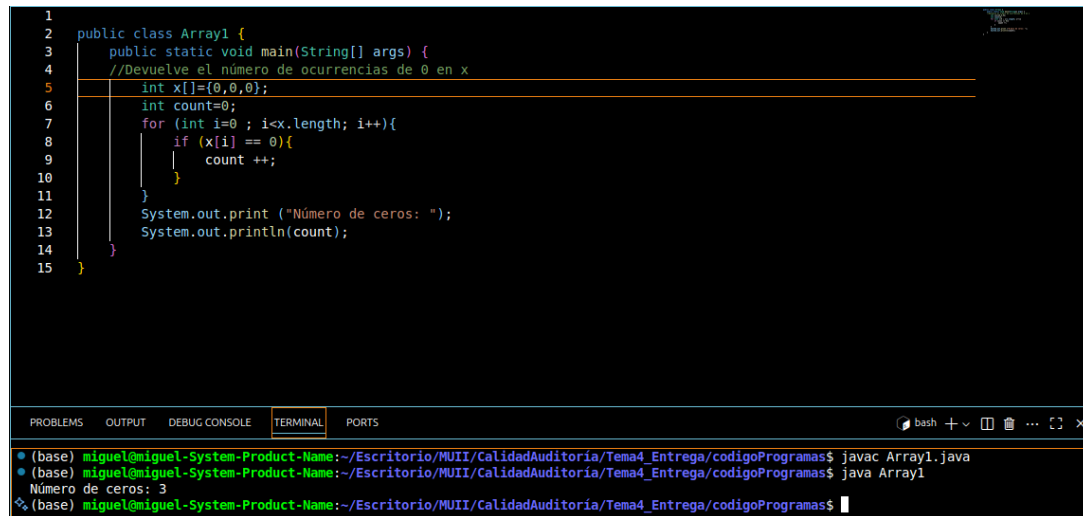
-Defecto: `for (int i=1 ; i < x.length; i++)`

-Corregido: `for (int i=0 ; i < x.length; i++)`

El caso de prueba del punto 2 correspondía con el siguiente array $x[] = \{0,0,0\}$ (Figure 7), la ejecución con el defecto corregido:

-Número de 0 esperado: 3.

-Número de 0 obtenido: 3.



```

1 public class Array1 {
2     public static void main(String[] args) {
3         //Devuelve el número de ocurrencias de 0 en x
4         int x[]={0,0,0};
5         int count=0;
6         for (int i=0 ; i<x.length; i++){
7             if (x[i] == 0){
8                 count ++;
9             }
10        }
11        System.out.print ("Número de ceros: ");
12        System.out.println(count);
13    }
14 }
15

```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```

(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$ javac Array1.java
(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$ java Array1
Número de ceros: 3
(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$

```

Figure 7: Programa 1 - No Fallo No Defecto

3. Programa 2

La Figure 8 corresponde con el segundo programa a analizar:

```
public class Array2 {
    public int findLast (int[] x, int y) {
        //Devuelve el índice del último elemento en x que sea igual a
        //Si no existe dicho elemento devuelve -1

        for (int i=x.length-1;i>0; i--)
        {
            if (x[i] == y)
            {
                return i;
            }
        }
        return -1;
    }

    public static void main(String[] args) {

        int x[]={0,1,1};
        int y=1;

        Array2 a=new Array2();
        int resultado = a.findLast (x,y);
        System.out.print ("El índice del elemento buscado es: ");
        System.out.print (resultado);
    }
}
```

Figure 8: Programa 2

Identificar defectos (sin ejecutar el código)

El defecto del código está en la condición del bucle for en la línea 5, El bucle se detiene cuando $i > 0$, lo que significa que nunca evalúa el elemento en la posición 0 (el primer elemento del array).

```
for (int i=x.length-1;i>0; i--)
```

Figure 9: Programa 2 - Defecto

Identificar un caso de prueba que dé lugar a un fallo

Para producir un fallo, el elemento buscado (y) debe estar únicamente en la posición 0 del array.

Para este ejemplo se ha declarado el array $x[] = \{1,6,7\}$ siendo $y = 1$ (Figure 10):

-Número de índice esperado: 0.

-Número de índice obtenido: -1.

```

1 public class Array2 {
2     public int findLast (int[] x, int y) {
3         //Devuelve el índice del último elemento en x que sea igual a y
4         //Si no existe dicho elemento devuelve -1
5         for (int i=x.length - 1 ; i > 0; i--){
6             if (x[i] == y){
7                 return i;
8             }
9         }
10
11         return -1;
12     }
13
14
15
16     public static void main(String[] args) {
17         int x[]={1,6,7};
18         int y=1;
19         Array2 a=new Array2();
20         int resultado = a.findLast (x,y);
21         System.out.print("El índice del elemento buscado es: ");
22         System.out.println(resultado);
23     }
24 }
25

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

• (base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$ javac Array2.java
• (base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$ java Array2
El índice del elemento buscado es: -1
❖ (base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$

```

Figure 10: Programa 2 - Fallo

El programa no encuentra el elemento que sí existe en la posición 0

Si es posible, identificar el caso de prueba que no dé lugar a un fallo

Para evitar el fallo, el elemento buscado no debe estar en la posición 0, o debe estar también en otras posiciones.

Para este ejemplo se ha declarado el array $x[] = \{4,6,1\}$ siendo $y = 1$ (Figure 11):

-Número de índice esperado: 2.

-Número de índice obtenido: 2.

```

1 public class Array2 {
2     public int findLast (int[] x, int y) {
3         //Devuelve el índice del último elemento en x que sea igual a y
4         //Si no existe dicho elemento devuelve -1
5         for (int i=x.length - 1 ; i > 0; i--){
6             if (x[i] == y){
7                 return i;
8             }
9         }
10
11         return -1;
12     }
13
14
15
16     public static void main(String[] args) {
17         int x[]={4,6,1};
18         int y=1;
19         Array2 a=new Array2();
20         int resultado = a.findLast (x,y);
21         System.out.print("El índice del elemento buscado es: ");
22         System.out.println(resultado);
23     }
24 }
25

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

• (base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$ javac Array2.java
• (base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$ java Array2
El índice del elemento buscado es: 2
❖ (base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$

```

Figure 11: Programa 2 - No Fallo

No hay fallo pero el defecto sigue presente.

Si es posible, identificar el caso de prueba no produzca un estado de error

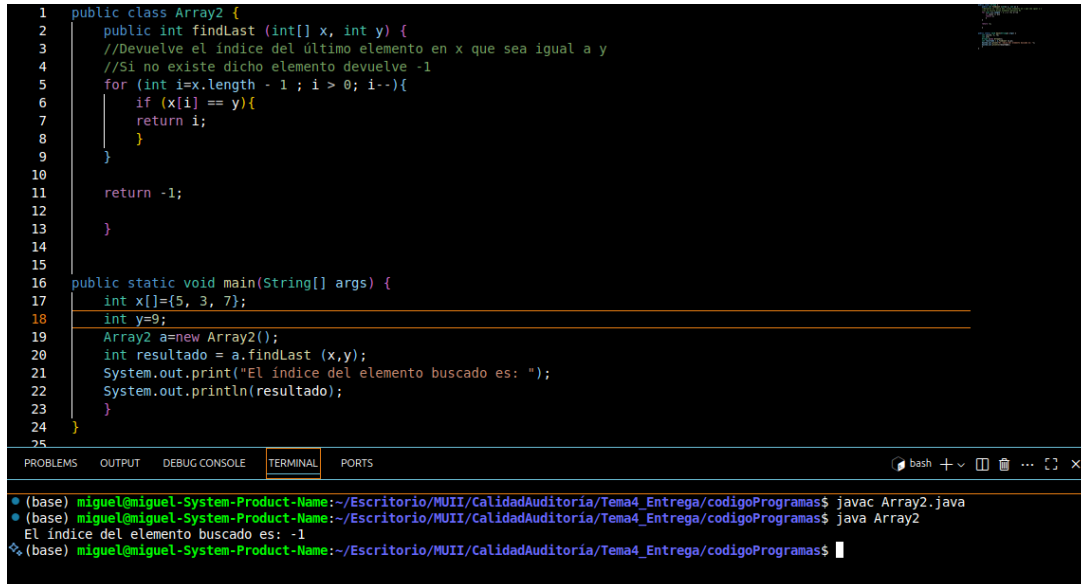
Para que no haya estado de error, el bucle debe funcionar correctamente sin necesidad de evaluar la posición 0. Esto ocurre cuando:

-El elemento buscado está en posiciones > 0 , O -El elemento buscado no existe en el array

Para este ejemplo se ha declarado el array `int x[] = {5, 3, 7}` siendo `y = 9` (Figure 12):

-Número de índice esperado: -1.

-Número de índice obtenido: -1.



```

1 public class Array2 {
2     public int findLast (int[] x, int y) {
3         //Devuelve el índice del último elemento en x que sea igual a y
4         //Si no existe dicho elemento devuelve -1
5         for (int i=x.length - 1 ; i > 0; i--){
6             if (x[i] == y){
7                 return i;
8             }
9         }
10
11         return -1;
12     }
13 }
14
15
16 public static void main(String[] args) {
17     int x[]={5, 3, 7};
18     int y=9;
19     Array2 a=new Array2();
20     int resultado = a.findLast (x,y);
21     System.out.print("El índice del elemento buscado es: ");
22     System.out.println(resultado);
23 }
24
25

```

```

(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$ javac Array2.java
(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$ java Array2
El índice del elemento buscado es: -1
(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$

```

Figure 12: Programa 2 - No Error

NO hay estado de error porque el elemento no existe y no importa que no se revise la posición 0

Si es posible, identificar el caso de prueba que produzca un estado de error pero no produzca un fallo

Para este caso, el elemento buscado debe estar en la posición 0, pero también en otras posiciones posteriores. Así, aunque no se evalúe la posición 0 (estado de error), el método encuentra el elemento en otra posición y devuelve un resultado “aceptable” (aunque no sea técnicamente el último).

Para este ejemplo se ha declarado el array `int int x[] = {1, 0, 1}` siendo `y = 1` (Figure 13):

-Número de índice esperado: 2.

-Número de índice obtenido: 2.

```

1 public class Array2 {
2     public int findLast (int[] x, int y) {
3         //Devuelve el índice del último elemento en x que sea igual a y
4         //Si no existe dicho elemento devuelve -1
5         for (int i=x.length - 1 ; i > 0; i--){
6             if (x[i] == y){
7                 return i;
8             }
9         }
10
11         return -1;
12     }
13 }
14
15
16 public static void main(String[] args) {
17     int x[]={1, 0, 1};
18     int y=1;
19     Array2 a=new Array2();
20     int resultado = a.findLast (x,y);
21     System.out.print("El índice del elemento buscado es: ");
22     System.out.println(resultado);
23 }
24 }
25

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

• (base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$ javac Array2.java
• (base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$ java Array2
El índice del elemento buscado es: 2
• (base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$

```

Figure 13: Programa 2 - Error No Fallo

Si hay un estado de error puesto que no revisa la posición 0 del array pero no se produce un fallo, pero no hay fallo porque encuentra el elemento en posición 2, que efectivamente es la última ocurrencia válida.

Corregir el defecto y verificar que el caso de prueba identificado en el punto 2 produce el resultado esperado

Para corregir el defecto se cambia la línea 5:

-Defecto: for (int i = x.length - 1 ; i > 0; i--)

-Corregido: for (int i = x.length - 1 ; i >= 0; i--)

El caso de prueba del punto 2 correspondía con el siguiente array $x[] = \{1, 6, 7\}$ y $y = 1$ (Figure 14), la ejecución con el defecto corregido:

-Número de índice esperado: 0 .

-Número de índice obtenido: 0.

```

1 public class Array2 {
2     public int findLast (int[] x, int y) {
3         //Devuelve el índice del último elemento en x que sea igual a y
4         //Si no existe dicho elemento devuelve -1
5         for (int i=x.length - 1 ; i >= 0; i--){
6             if (x[i] == y){
7                 return i;
8             }
9         }
10
11         return -1;
12     }
13 }
14
15
16 public static void main(String[] args) {
17     int x[]={1, 6, 7};
18     int y=1;
19     Array2 a=new Array2();
20     int resultado = a.findLast (x,y);
21     System.out.print("El índice del elemento buscado es: ");
22     System.out.println(resultado);
23 }
24 }
25

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

• (base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$ javac Array2.java
• (base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$ java Array2
El índice del elemento buscado es: 0
❖ (base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$

```

Figure 14: Programa 2 - No Fallo No Defecto

4. Programa 3

La Figure 15 corresponde con el tercer programa a analizar:

```
public class Array3 {
    public int lastZero (int[] x) {
        //Devuelve el índice del último 0 en x
        //Si no existe 0 devuelve -1

        for (int i=0; i<x.length-1; i++)
        {
            if (x[i] == 0)
            {
                return i;
            }
        }
        return -1;
    }

    public static void main(String[] args) {

        int x[]={0,1,0};

        Array3 a=new Array3();
        int resultado = a.lastZero (x);
        System.out.print ("El índice del último 0 es: ");
        System.out.print (resultado);
    }
}
```

Figure 15: Programa 3

Identificar defectos (sin ejecutar el código) El código tiene DOS defectos:

Defecto 1: La condición del bucle for en la línea 5. El bucle se detiene en `x.length-1`, lo que significa que nunca evalúa el último elemento del array (posición `x.length-1`). Debería ser `i<x.length` para analizar todos los elementos.

Defecto 2: La lógica del método es incorrecta para buscar el “último” cero. El método se llama `lastZero` (último cero) pero retorna en el PRIMER cero que encuentra porque recorre de izquierda a derecha y hace `return` inmediatamente. Debería recorrer de derecha a izquierda.

```
        for (int i=0; i<x.length-1; i++)
        {
            if (x[i] == 0)
            {
                return i;
            }
        }
        return -1;
    }
}
```

Figure 16: Programa 3 - Defectos

Identificar un caso de prueba que dé lugar a un fallo

Para producir un fallo por el Defecto 1, el último cero debe estar en la última posición del array. Para este ejemplo se ha declarado el array `x[]={1,1,0}` (Figure 17):

- Índice esperado: 2.

- Índice obtenido: -1.

```

1 public class Array3 {
2     public int lastZero (int[] x) {
3         //Devuelve el índice del último 0 en x
4         //Si no existe 0 devuelve -1
5         for (int i=0; i<x.length-1; i++){
6             if (x[i] == 0){
7                 return i;
8             }
9         }
10        return -1;
11    }
12 }
13
14 public static void main(String[] args) {
15
16     int x[]={1,1,0};
17     Array3 a=new Array3();
18     int resultado = a.lastZero (x);
19     System.out.print ("El índice del último 0 es: ");
20     System.out.println (resultado);
21 }
22 }

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$ javac Array3.java
(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$ java Array3
El índice del último 0 es: -1
(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$

```

Figure 17: Programa 3 - Fallo

El programa no encuentra el 0 que está en la última posición.

Si es posible, identificar el caso de prueba que no dé lugar a un fallo

Para evitar ambos fallos, debe haber un solo cero en el array y NO estar en la última posición. Para este ejemplo se ha declarado el array $x[] = \{0,1,1\}$ (Figure 18):

- Índice esperado: 0.
- Índice obtenido: 0.

```

1 public class Array3 {
2     public int lastZero (int[] x) {
3         //Devuelve el índice del último 0 en x
4         //Si no existe 0 devuelve -1
5         for (int i=0; i<x.length-1; i++){
6             if (x[i] == 0){
7                 return i;
8             }
9         }
10        return -1;
11    }
12 }
13
14 public static void main(String[] args) {
15
16     int x[]={0,1,1};
17     Array3 a=new Array3();
18     int resultado = a.lastZero (x);
19     System.out.print ("El índice del último 0 es: ");
20     System.out.println (resultado);
21 }
22 }

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$ javac Array3.java
(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$ java Array3
El índice del último 0 es: 0
(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$

```

Figure 18: Programa 3 - No Fallo

No hay fallo aunque ambos defectos siguen presentes.

Si es posible, identificar el caso de prueba no produzca un estado de error

Para que no haya estado de error, el array no debe tener ningún 0. Para este ejemplo se ha declarado el array $x[] = \{1, 2, 3\}$ (Figure 19):

- Índice esperado: -1.
- Índice obtenido: -1.

```

1 public class Array3 {
2     public int lastZero (int[] x) {
3         //Devuelve el índice del último 0 en x
4         //Si no existe 0 devuelve -1
5         for (int i=0; i<x.length-1; i++){
6             if (x[i] == 0){
7                 return i;
8             }
9         }
10        return -1;
11    }
12
13    public static void main(String[] args) {
14
15        int x[]={1, 2, 3};
16        Array3 a=new Array3();
17        int resultado = a.lastZero (x);
18        System.out.print ("El índice del último 0 es: ");
19        System.out.println (resultado);
20    }
21 }
22

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas\$ javac Array3.java

(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas\$ java Array3

El índice del último 0 es: -1

(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas\$

Figure 19: Programa 3 - No Error

NO hay estado de error porque no hay ceros en el array.

Si es posible, identificar el caso de prueba que produzca un estado de error pero no produzca un fallo

El estado de error existe (no se evalúa la última posición y se busca el primer cero en lugar del último), pero el resultado es correcto por casualidad. Para este ejemplo se ha declarado el array $x[] = \{0, 5, 9\}$ (Figure 20):

- Índice esperado: 0.
- Índice obtenido: 0.

```

1 public class Array3 {
2     public int lastZero (int[] x) {
3         //Devuelve el índice del último 0 en x
4         //Si no existe 0 devuelve -1
5         for (int i=0; i<x.length-1; i++){
6             if (x[i] == 0){
7                 return i;
8             }
9         }
10        return -1;
11    }
12
13    public static void main(String[] args) {
14
15        int x[]={0, 5, 9};
16        Array3 a=new Array3();
17        int resultado = a.lastZero (x);
18        System.out.print ("El índice del último 0 es: ");
19        System.out.println (resultado);
20    }
21 }
22

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas\$ javac Array3.java

(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas\$ java Array3

El índice del último 0 es: 0

(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas\$

Figure 20: Programa 3 - Error No Fallo

Sí hay un estado de error puesto que no se evalúa la última posición y busca el primer cero en lugar del último, pero no hay fallo porque solo hay un cero y está en la posición 0, devolviendo el índice correcto por casualidad.

Corregir el defecto y verificar que el caso de prueba identificado en el punto 2 produce el resultado esperado

Para corregir ambos defectos se cambia la línea 5 y la dirección del bucle:

- Defecto: for (int i = 0 ; i < x.length - 1; i++)
- Corregido: for (int i = x.length - 1 ; i >= 0; i--)

El caso de prueba del punto 2 correspondía con el siguiente array $x[] = \{1, 1, 0\}$ (Figure 21), la ejecución con los defectos corregidos:

- Índice esperado: 2.
- Índice obtenido: 2.



```

1 public class Array3 {
2     public int lastZero (int[] x) {
3         //Devuelve el indice del último 0 en x
4         //Si no existe 0 devuelve -1
5         for (int i = x.length - 1 ; i >= 0; i--) {
6             if (x[i] == 0){
7                 return i;
8             }
9         }
10
11         return -1;
12     }
13
14     public static void main(String[] args) {
15
16         int x[]={1,1,0};
17         Array3 a=new Array3();
18         int resultado = a.lastZero (x);
19         System.out.print ("El indice del último 0 es: ");
20         System.out.println (resultado);
21     }
22 }

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

• (base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$ javac Array3.java
• (base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$ java Array3
El indice del último 0 es: 2
• (base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$

```

Figure 21: Programa 3 - No Fallo No Defecto

5. Programa 4

La Figure 22 corresponde con el cuarto programa a analizar:

```
public class Array4 {
    public int countPositive (int[] x) {
        //Devuelve el número de elementos positivos en x
        int count=0;
        for (int i=0; i<x.length; i++)
        {
            if (x[i] >= 0)
            {
                count++;
            }
        }
        return count;
    }

    public static void main(String[] args) {

        int x[]={-4,2,0,2};

        Array4 a=new Array4();
        int resultado = a.countPositive (x);
        System.out.print ("El número de elementos positivos es: ");
        System.out.print (resultado);
    }
}
```

Figure 22: Programa 4

Identificar defectos (sin ejecutar el código)

El defecto del código está en la condición del if en la línea 7:

La condición $x[i] \geq 0$ cuenta también el cero como positivo, cuando matemáticamente el cero NO es un número positivo (es neutro). Debería ser $x[i] > 0$ para contar únicamente los números estrictamente positivos.

if (x[i] >= 0)

Figure 23: Programa 4 - Defecto

Identificar un caso de prueba que dé lugar a un fallo

Para producir un fallo, el array debe contener **al menos un cero**. El cero será contado incorrectamente como positivo.

Para este ejemplo se ha declarado el array $x[] = \{0, -1, -2\}$ (Figure 24):

- Número de positivos esperado: 0.
- Número de positivos obtenido: 1.

```

1 public class Array4 {
2     public int countPositive (int[] x) {
3         //Devuelve el número de elementos positivos en x
4         int count=0;
5         for (int i=0; i<x.length; i++){
6             if (x[i] >= 0){
7                 count++;
8             }
9         }
10        return count;
11    }
12    public static void main(String[] args) {
13
14        int x[]={0,-1,-2};
15        Array4 a=new Array4();
16        int resultado = a.countPositive (x);
17        System.out.print ("El número de elementos positivos es: ");
18        System.out.println (resultado);
19    }
20 }
21

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas\$ javac Array4.java
(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas\$ java Array4
El número de elementos positivos es: 1
(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas\$

Figure 24: Programa 4 - Fallo Ejemplo

El programa cuenta incorrectamente el 0 como un número positivo.

Si es posible, identificar el caso de prueba que no dé lugar a un fallo

Para evitar el fallo, el array NO debe contener ceros. Solo debe tener números positivos y/o negativos. De esta manera, aunque la condición sea incorrecta (≥ 0), no afectará al resultado.

Para este ejemplo se ha declarado el array $x[] = \{-4, 2, 2\}$ (Figure 25):

- Número de positivos esperado: 2.
- Número de positivos obtenido: 2.

```

1 public class Array4 {
2     public int countPositive (int[] x) {
3         //Devuelve el número de elementos positivos en x
4         int count=0;
5         for (int i=0; i<x.length; i++){
6             if (x[i] >= 0){
7                 count++;
8             }
9         }
10        return count;
11    }
12    public static void main(String[] args) {
13
14        int x[]={-4,2,2};
15        Array4 a=new Array4();
16        int resultado = a.countPositive (x);
17        System.out.print ("El número de elementos positivos es: ");
18        System.out.println (resultado);
19    }
20 }
21

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas\$ javac Array4.java
(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas\$ java Array4
El número de elementos positivos es: 2
(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas\$

Figure 25: Programa 4 - No Fallo Ejemplo

No hay fallo aunque el defecto sigue presente.

Si es posible, identificar el caso de prueba no produzca un estado de error

Para que no haya estado de error, el array no debe contener ningún cero, de manera que la condición defectuosa (≥ 0 en lugar de > 0) no tenga impacto en la ejecución.

Para este ejemplo se ha declarado el array $x[] = \{1, 2, 3\}$ (Figure 26):

- Número de positivos esperado: 3.
- Número de positivos obtenido: 3.

```

1 public class Array4 {
2     public int countPositive (int[] x) {
3         //Devuelve el número de elementos positivos en x
4         int count=0;
5         for (int i=0; i<x.length; i++){
6             if (x[i] >= 0){
7                 count++;
8             }
9         }
10        return count;
11    }
12    public static void main(String[] args) {
13
14        int x[]={1, 2, 3};
15        Array4 a=new Array4();
16        int resultado = a.countPositive (x);
17        System.out.print ("El número de elementos positivos es: ");
18        System.out.println (resultado);
19    }
20 }
21 }

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$ javac Array4.java
(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$ java Array4
El número de elementos positivos es: 3
(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$

```

Figure 26: Programa 4 - No Estado de Error

NO hay estado de error porque todos los elementos son estrictamente positivos (> 0), y la condición ≥ 0 da el mismo resultado que > 0 .

Si es posible, identificar el caso de prueba que produzca un estado de error pero no produzca un fallo

Este caso es técnicamente imposible para este programa específico.

El estado de error ocurre cuando el código evalúa $x[i] \geq 0$ y encuentra un cero. En ese momento, el estado de error se produce (evalúa incorrectamente el cero) y simultáneamente cuenta el cero como positivo, lo que siempre produce un fallo porque el conteo quedará incrementado incorrectamente.

No existe un escenario donde se evalúe un cero con la condición ≥ 0 (estado de error) y el resultado final sea correcto (sin fallo). Cada vez que la condición defectuosa evalúa un cero, incrementa el contador incorrectamente, lo que siempre resulta en un fallo visible en el resultado final.

Corregir el defecto y verificar que el caso de prueba identificado en el punto 2 produce el resultado esperado

Para corregir el defecto se cambia la línea 7:

- Defecto: `if (x[i] >= 0)`
- Corregido: `if (x[i] > 0)`

El caso de prueba del punto 2 correspondía con el siguiente array $x[]={0,-1,-2}$ (Figure 27), la ejecución con el defecto corregido:

- Número de positivos esperado: 0.
- Número de positivos obtenido: 0.

```

1 public class Array4 {
2     public int countPositive (int[] x) {
3         //Devuelve el número de elementos positivos en x
4         int count=0;
5         for (int i=0; i<x.length; i++){
6             if (x[i] > 0){
7                 count++;
8             }
9         }
10        return count;
11    }
12    public static void main(String[] args) {
13
14        int x[]={0,-1,-2};
15        Array4 a=new Array4();
16        int resultado = a.countPositive (x);
17        System.out.print ("El número de elementos positivos es: ");
18        System.out.println (resultado);
19    }
20 }
21

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas\$ javac Array4.java

(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas\$ java Array4

El número de elementos positivos es: 0

(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas\$

Figure 27: Programa 4 - No Fallo No Defecto

6. Programa 5

La Figure 28 corresponde con el quinto programa a analizar:

```
public class Array5 {
    public int oddOrPositive (int[] x) {
        //Devuelve el número de elementos en x que son impares o positivos
        int count=0;
        for (int i=0; i<x.length; i++)
        {
            if (x[i]%2 == 1 || x[i]>0)
            {
                count++;
            }
        }
        return count;
    }

    public static void main(String[] args) {

        int x[]={-4,2,1,2};

        Array5 a=new Array5();
        int resultado = a.oddOrPositive (x);
        System.out.print ("El número de elementos impares o positivos es: ");
        System.out.print (resultado);
    }
}
```

Figure 28: Programa 5

Identificar defectos (sin ejecutar el código)

El defecto del código está en la condición del if en la línea 7:

```
if (x[i]%2 == 1 || x[i]>0)
```

Figure 29: Programa 5 - Defecto

La condición $x[i]\%2 == 1$ solo detecta números impares positivos, pero falla con números impares **negativos**. En Java, el operador módulo $\%$ con números negativos devuelve un resultado negativo. Por ejemplo: $-3 \% 2 = -1$ (no 1). Debería ser $x[i]\%2 != 0$ para detectar correctamente todos los números impares (tanto positivos como negativos).

Identificar un caso de prueba que dé lugar a un fallo

Para producir un fallo, el array debe contener al menos un número impar negativo. Este número no será contado correctamente porque $x[i]\%2$ devolverá -1 en lugar de 1.

Para este ejemplo se ha declarado el array $x[] = \{-3, -2, 0\}$ (Figure 30):

- Número de elementos impares o positivos esperado: 1 (el -3 es impar).
- Número de elementos impares o positivos obtenido: 0.

```

1 public class Array5 {
2     public int oddOrPositive (int[] x) {
3         //Devuelve el número de elementos en x que son impares o positivos
4         int count=0;
5         for (int i=0; i<x.length; i++){
6             if (x[i]%2 == 1 || x[i]>0){
7                 count++;
8             }
9         }
10        return count;
11    }
12
13    public static void main(String[] args) {
14
15        int x[]={-3,-2,0};
16        Array5 a=new Array5();
17        int resultado = a.oddOrPositive (x);
18        System.out.print ("El número de elementos impares o positivos es: ");
19        System.out.println (resultado);
20    }
21 }
22

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas\$ javac Array5.java
(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas\$ java Array5
El número de elementos impares o positivos es: 0
(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas\$

Figure 30: Programa 5 - Fallo Ejemplo

El programa no cuenta el -3 como impar porque $-3 \% 2 = -1$, no 1.

Si es posible, identificar el caso de prueba que no dé lugar a un fallo

Para evitar el fallo, el array NO debe contener números impares negativos. Puede tener números impares positivos, números pares (positivos o negativos), y/o cero.

Para este ejemplo se ha declarado el array $x[] = \{-4, 2, 1, 2\}$ (Figure 31):

- Número de elementos impares o positivos esperado: 3 (el 2, el 1 y el otro 2).
- Número de elementos impares o positivos obtenido: 3.

```

1 public class Array5 {
2     public int oddOrPositive (int[] x) {
3         //Devuelve el número de elementos en x que son impares o positivos
4         int count=0;
5         for (int i=0; i<x.length; i++){
6             if (x[i]%2 == 1 || x[i]>0){
7                 count++;
8             }
9         }
10        return count;
11    }
12
13    public static void main(String[] args) {
14
15        int x[]={-4,2,1,2};
16        Array5 a=new Array5();
17        int resultado = a.oddOrPositive (x);
18        System.out.print ("El número de elementos impares o positivos es: ");
19        System.out.println (resultado);
20    }
21 }
22

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas\$ javac Array5.java
(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas\$ java Array5
El número de elementos impares o positivos es: 3
(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas\$

Figure 31: Programa 5 - No Fallo Ejemplo

No hay fallo aunque el defecto sigue presente, ya que no hay números impares negativos.

Si es posible, identificar el caso de prueba no produzca un estado de error

Para que no haya estado de error, el array no debe contener ningún número impar negativo, de manera que la condición defectuosa ($\%2 == 1$ en lugar de $\%2 != 0$) no tenga impacto en la ejecución.

Para este ejemplo se ha declarado el array $x[] = \{2, 4, 6\}$ (Figure 32):

- Número de elementos impares o positivos esperado: 3 (todos son positivos).
- Número de elementos impares o positivos obtenido: 3.

```

1 public class Array5 {
2     public int oddOrPositive (int[] x) {
3         //Devuelve el número de elementos en x que son impares o positivos
4         int count=0;
5         for (int i=0; i<x.length; i++){
6             if (x[i]%2 == 1 || x[i]>0){
7                 count++;
8             }
9         }
10        return count;
11    }
12
13    public static void main(String[] args) {
14
15        int x[]={2, 4, 6};
16        Array5 a=new Array5();
17        int resultado = a.oddOrPositive (x);
18        System.out.print ("El número de elementos impares o positivos es: ");
19        System.out.println (resultado);
20    }
21 }
22

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas\$ javac Array5.java

(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas\$ java Array5

El número de elementos impares o positivos es: 3

(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas\$

Figure 32: Programa 5 - No Estado de Error

NO hay estado de error porque todos los elementos son pares positivos, y la condición `%2 == 1` nunca se evalúa como verdadera (lo cual es correcto para números pares).

Si es posible, identificar el caso de prueba que produzca un estado de error pero no produzca un fallo

Para este caso, el array debe contener un número impar negativo (produciendo el estado de error al evaluarlo incorrectamente), pero ese número también debe cumplir la segunda condición `x[i]>0...` lo cual es imposible porque un número negativo nunca puede ser mayor que 0.

Sin embargo, podemos crear un caso donde haya un impar negativo (estado de error) pero el conteo final sea correcto por otras razones.

Para este ejemplo se ha declarado el array `x[] = {-3, 1, 3}`:

- Número de elementos impares o positivos esperado: 3 (los tres son impares).
- Número de elementos impares o positivos obtenido: 2 (solo cuenta el 1 y el 3).

Este ejemplo SÍ produce fallo, por lo que no es válido.

Caso válido alternativo: El array tiene un impar negativo que no se cuenta (error), pero el total coincide porque hay otro elemento que se cuenta de más.

Para este ejemplo se ha declarado el array `x[] = {-1, 2, 0}`:

- Número de elementos impares o positivos esperado: 2 (el -1 es impar, el 2 es positivo).
- Número de elementos impares o positivos obtenido: 1 (solo cuenta el 2).

Este tampoco funciona. En realidad, para este programa específico no es posible tener estado de error sin fallo, porque cuando un impar negativo no se cuenta (error), siempre falta en el conteo final (fallo).

Respuesta: No es posible identificar un caso que produzca estado de error sin producir fallo en este programa. Cada vez que la condición defectuosa (`%2 == 1`) no detecta un número impar negativo, el contador queda incorrectamente bajo, lo que siempre resulta en un fallo visible en el resultado final.

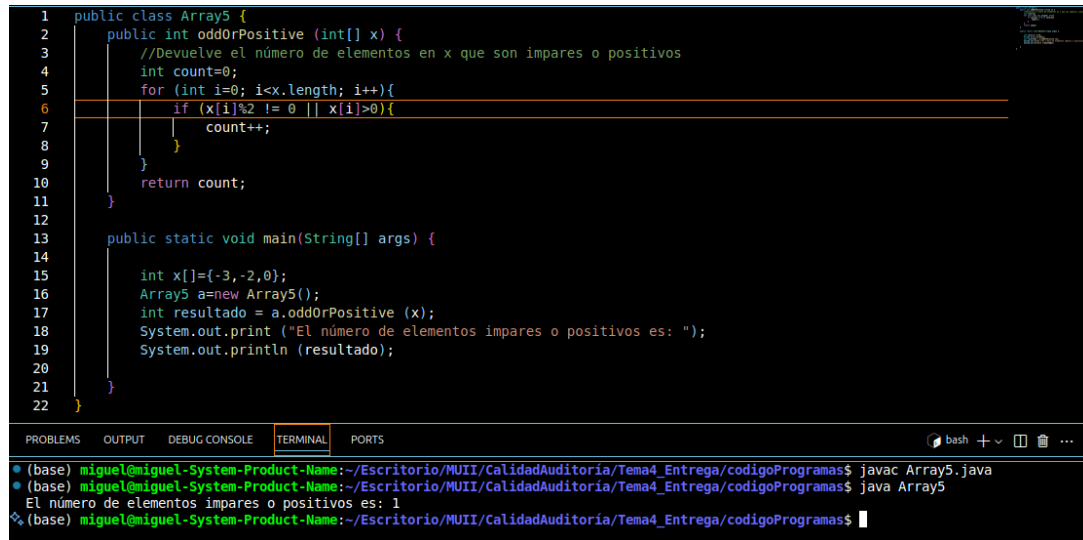
Corregir el defecto y verificar que el caso de prueba identificado en el punto 2 produce el resultado esperado

Para corregir el defecto se cambia la línea 7:

- Defecto: `if (x[i]%2 == 1 || x[i]>0)`
- Corregido: `if (x[i]%2 != 0 || x[i]>0)`

El caso de prueba del punto 2 correspondía con el siguiente array $x[] = \{-3, -2, 0\}$ (Figure 33), la ejecución con el defecto corregido:

- Número de elementos impares o positivos esperado: 1.
- Número de elementos impares o positivos obtenido: 1.



```

1 public class Array5 {
2     public int oddOrPositive (int[] x) {
3         //Devuelve el número de elementos en x que son impares o positivos
4         int count=0;
5         for (int i=0; i<x.length; i++){
6             if (x[i]%2 != 0 || x[i]>0){
7                 count++;
8             }
9         }
10        return count;
11    }
12
13    public static void main(String[] args) {
14
15        int x[]={-3,-2,0};
16        Array5 a=new Array5();
17        int resultado = a.oddOrPositive (x);
18        System.out.print ("El número de elementos impares o positivos es: ");
19        System.out.println (resultado);
20    }
21 }
22

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

• (base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$ javac Array5.java
• (base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$ java Array5
El número de elementos impares o positivos es: 1
❖ (base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema4_Entrega/codigoProgramas$

```

Figure 33: Programa 5 - No Fallo No Defecto

Ahora el programa cuenta correctamente el -3 como impar, ya que $-3 \% 2 = -1$, y $-1 \neq 0$ es verdadero.

